

Toward a Taxonomy of Vulnerability Patterns in AI-Assistant-Generated Code

Jitendra Rout

Independent Researcher

[email/GitHub link]

September 2026

Abstract

AI coding assistants have become a routine part of software development, and a growing body of anecdotal and industry commentary suggests they introduce security vulnerabilities in patterns distinct from those found in human-authored code. This paper proposes a preliminary taxonomy of seven such patterns, grounded in a proposed causal account of why each pattern arises from the incentives of code-sample generation rather than from model capability limits. We implement a proof-of-concept static analysis scanner, `ai-vuln-scan`, targeting these patterns, and report an empirical comparison against two established open-source static analysis tools (Bandit for Python, and the `eslint-plugin-security` plugin for JavaScript) on a small hand-constructed test corpus. We find that general-purpose security linters, run with their default offline rule sets, miss a substantial share of the patterns in our corpus — most notably a class of inter-route authorization-consistency bug with no direct analogue in either baseline tool. We report these results candidly, including a case where our own tool underperformed the established baseline, and outline the taxonomy, tooling, and evaluation methodology as an open, extensible artifact for further community contribution.

1 Introduction

Since the widespread adoption of AI coding assistants (GitHub Copilot, Claude Code, and comparable tools), a recurring concern in developer and security communities is that AI-generated code exhibits security defects at a rate, and in a distribution, that differs meaningfully from human-authored code. This concern is largely anecdotal in the public discourse — security newsletters and conference talks routinely cite examples — but there is comparatively little open, structured tooling built specifically around the premise that these are *distinct, nameable* patterns rather than a generic increase in defect rate.

This paper makes three contributions:

1. A taxonomy of seven vulnerability patterns (Section 3), each with a proposed causal account of *why* the pattern is disproportionately associated with AI-assisted generation, not merely possible in any code.
2. An open-source reference implementation, `ai-vuln-scan` (Section 4), that detects instances of six of the seven patterns using structural and pattern-based heuristics.
3. An empirical comparison (Section 5) against two established, independently-maintained static analysis tools on a small test corpus, reported with full honesty about cases where our tool under- or over-performed the baseline.

We emphasize at the outset that this is preliminary, small-sample work – a proposed taxonomy and a proof-of-concept scanner, not a large-scale empirical study of real-world AI-generated code. We see the primary contribution as the taxonomy and the open, versioned, contribution-friendly structure in which it is published, rather than a claim of comprehensive tool performance.

2 Related Work

General-purpose static analysis tools for security — Bandit [1] for Python, `eslint-plugin-security` [2] for JavaScript, and commercial/hosted tools such as Semgrep [3] — detect a broad range of vulnerability classes using pattern matching, taint analysis, and (for some tools) hosted rule registries. These tools are not designed around any particular hypothesis about *generation source*; a hardcoded secret is flagged identically whether written by a human or an AI assistant. Our work is complementary rather than competitive: we do not claim to detect a broader class of vulnerabilities than these tools in general, only that a subset of patterns is worth naming and tuning for specifically because of their disproportionate association with AI-assisted generation.

Industry discussion of “AI-introduced” vulnerabilities is active but largely takes the form of blog posts, vendor marketing content, and conference talks rather than a shared, citable taxonomy with an open reference implementation. We are not aware of a prior open-source project structured explicitly around a versioned, contribution-open pattern catalog of this kind, though we note the field is moving quickly and invite correction via the project’s issue tracker.

3 A Taxonomy of AI-Associated Vulnerability Patterns

We propose the following seven patterns. For each, we state the pattern, a proposed causal account of why it is disproportionately associated with AI-assisted code generation specifically (as opposed to being simply possible in any code), and our detection approach.

AIVP-001: Hardcoded Secret in Example/Placeholder Form

A plausible-looking example credential (API key, connection string) is generated to make a code sample runnable, then copy-pasted into real code without substitution. *Detection*: known-key-format regular expressions (AWS, Stripe, GitHub token shapes) combined with Shannon entropy scoring on string literals assigned to secret-like variable names.

AIVP-002: Unparameterized Query Construction

String interpolation into a query is the more narratively direct way to express “build a query using this variable” than explaining parameterization. *Detection*: pattern matching on query-execution calls containing template-literal or f-string interpolation adjacent to a SQL keyword.

AIVP-003: Shell Command Built by String Interpolation

Analogous to AIVP-002: `exec("cmd " + arg)` reads as the intuitive example; an argument-array-based safe call is correct but a less frequently generated default. *Detection*: pattern matching on shell-execution calls with concatenated or interpolated arguments.

AIVP-004: Permissive Default Configuration

Wildcard CORS origins, disabled TLS verification, and debug flags left enabled reduce setup friction in a self-contained example, where the model lacks the real-world context (actual allowed origins, a real certificate) needed to generate the hardened equivalent. *Detection*: direct pattern matching on known insecure configuration flags.

AIVP-005: Weak Cryptographic Primitive in a Security Context

MD5/SHA1 and non-cryptographic random sources (`Math.random()`) are frequently the first primitive associated with a generic “hash this” or “generate a random string” request; the security-context distinction is not always surfaced absent an explicit prompt. *Detection*: pattern matching on weak-primitive calls adjacent to security-relevant identifiers (`password`, `token`, `secret`).

AIVP-006: Inconsistent Authorization Across Near-Identical Routes

We consider this the most distinctly AI-associated pattern in the set. When an assistant is asked to add a route “like the others,” it regenerates the handler from a natural-language description rather than duplicating the existing code block; regeneration is where a previously-consistent step (an authorization middleware call) can silently drop, particularly across separate prompts or edits. We propose this differs structurally from a typical human-introduced authorization bug, where copy-paste duplication of an existing block tends to *preserve* the full pattern by construction. *Detection*: for a given file, identify route-defining calls, determine the middleware identifier applied to a majority of routes, and flag routes lacking it.

AIVP-007: Verbose Error Response Leaking Internals

Returning a raw exception object or stack trace directly in an HTTP response is the fastest way to make error handling visibly “work” during iterative generation; separating server-side logging from a generic client-facing message is a production concern not always present in a first-pass generation. *Detection*: pattern matching on error objects or stack-trace values passed directly into response-sending calls.

The full catalog, including contribution guidelines for proposing additional patterns independent of whether a detection rule yet exists, is maintained openly at the project repository.

4 Reference Implementation

We implement `ai-vuln-scan`, an open-source (MIT-licensed) command-line scanner covering AIVP-001 through AIVP-005 and AIVP-007 via regular-expression and structural pattern matching, and AIVP-006 via a lightweight route-extraction and majority-middleware heuristic (Section 3). The tool supports JavaScript/TypeScript and Python, integrates as a CI gate or pre-commit hook (non-zero exit on any critical-severity finding), and ships with a labeled example corpus (Section 5) usable as a regression fixture.

We explicitly do not implement full abstract-syntax-tree-based taint analysis in this version; detection is regex- and structural-heuristic-based, which we discuss as a limitation in Section 6.

5 Evaluation

5.1 Corpus

We constructed a small hand-written test corpus of two files (one JavaScript/Express, one Python/Flask) deliberately containing instances of each pattern in Section 3, modeled on realistic AI-assistant output rather than contrived edge cases. A third file contains the same routes rewritten to be free of these issues, used to check for false positives. The full corpus is published alongside the tool. We are explicit that this is a small, hand-constructed corpus, not a random or representative sample of real-world AI-generated code; the evaluation below should be read as a demonstration of feasibility and a starting comparison point, not a general performance claim.

5.2 Baselines

We compare against two established, independently maintained static analysis tools, run with their default, fully offline configurations:

- **Bandit** [1] (v1.9), a widely used Python security linter, run against the Python file in our corpus.
- **eslint-plugin-security** [2] (v4.0), a widely used JavaScript security rule plugin for ESLint, run against the JavaScript file in our corpus with its full rule set enabled.

We note that Semgrep [3], arguably the most prominent tool in this space, ships without a bundled offline rule set — its default usage mode fetches rules from a hosted registry, which was not accessible in our evaluation environment. We report this as a methodological limitation rather than omit Semgrep silently; a network-enabled replication comparing against Semgrep’s registry rule sets is noted as future work.

5.3 Results

Pattern	ai-vuln-scan	Bandit	eslint-plugin-security
AIVP-001 (hardcoded secret)	✓ (both files)	✗	✗
AIVP-002 (unparam. query, direct)	✓ (JS)	—	✗
AIVP-002 (unparam. query, via variable)	✗ (Python, missed)	✓ (Python)	—
AIVP-003 (shell interpolation)	✓ (both)	✓ (Python)	✗
AIVP-004 (permissive defaults)	✓ (both)	✓ (Python, partial)	✗
AIVP-005 (weak crypto/random)	✓ (both)	✓ (Python, MD5 only)	✗
AIVP-006 (auth inconsistency)	✓ (JS)	—	✗ (no analogue)
AIVP-007 (error leakage)	✓ (Python)	✗	—
False positives (clean file)	0	—	0

Table 1: Detection results by pattern. ✓ = detected, ✗ = missed, — = pattern not applicable to that tool’s target language in our corpus. AIVP-002 is split into two rows because our tool’s regex-based approach detects the pattern when the tainted string is passed directly to a query call but — a limitation we report candidly — misses the case where the same tainted string is first assigned to an intermediate variable, which Bandit’s more sophisticated analysis catches correctly.

Across our corpus, `ai-vuln-scan` produced 13 findings (6 critical, 4 high, 3 medium severity) with zero false positives against the clean comparison file. Bandit produced 7 findings against the Python file, correctly identifying 4 of 6 patterns we targeted plus one (unparameterized-query-via-variable) that our tool missed, alongside two findings (subprocess-import advisory, missing-request-timeout) outside the scope of

our taxonomy entirely. `eslint-plugin-security`, run with its full default rule set, produced zero findings against our JavaScript file — consistent with that plugin’s design focus on a different vulnerability class (unsafe regular expressions, non-literal filesystem/require calls, timing attacks) rather than the patterns in our taxonomy.

5.4 Discussion

Two results are worth highlighting. First, the AIVP-006 (authorization-inconsistency) pattern has no analogue in either baseline tool’s detection surface — neither tool reasons about consistency across multiple route definitions within a file. We view this as modest evidence that at least one pattern in our taxonomy identifies a genuinely under-served detection surface, though we caution against generalizing from a single corpus instance.

Second, and reported candidly rather than omitted: our tool missed a SQL-injection instance that Bandit correctly caught, because our detection logic requires the tainted string literal to appear directly in the query-call argument, and does not trace data flow through an intermediate variable assignment. This is a direct consequence of using regex/structural heuristics rather than proper taint analysis, and we discuss it further in Section 6. We consider it important to report this result rather than select only favorable comparisons, both as good methodological practice and because the honest gap is itself informative: it demonstrates precisely why a production-grade version of this tool would need AST-based taint tracking rather than pattern matching alone.

6 Limitations

- **Corpus size and construction.** Our evaluation corpus is small and hand-constructed by the authors, not sampled from real-world AI-generated commits. It should be read as a feasibility demonstration, not a claim of general detection performance. A follow-up study using a larger corpus of real, labeled AI-assistant commit diffs is the clearest next step and is left as future work.
- **Detection approach.** Regex/structural heuristics are precise for the patterns they target but will miss variants involving intermediate variables, aliasing, or multi-file data flow, as demonstrated in Section 5. A taint-analysis-based successor is planned.
- **Semgrep comparison incomplete.** As noted, we could not evaluate against Semgrep’s hosted registry rule sets in our environment; this comparison is left for a network-enabled replication.
- **Causal claims are proposed, not directly tested.** The “why” explanations in Section 3 are plausible accounts consistent with the patterns observed, not the product of a controlled study isolating AI-assistant generation behavior against a human-authored control group. We regard this as the most important open question for follow-up work: a systematic comparison of matched human- and AI-authored code samples would allow these causal claims to be tested directly rather than proposed.

7 Conclusion

We present a preliminary, seven-pattern taxonomy of vulnerabilities disproportionately associated with AI-assistant-generated code, an open-source reference scanner implementing detection for six of the seven patterns, and a small empirical comparison against two established baseline tools. We report our results candidly, including a case where our tool underperformed an established baseline, and frame the taxonomy itself — published as an open, versioned, contribution-welcoming artifact independent of any particular

tool’s implementation — as the primary contribution of this work. We hope this provides a starting point for more rigorous, larger-scale study of this problem by the broader security research community.

Availability

The taxonomy, reference implementation, and evaluation corpus described in this paper are published under the MIT license at <https://github.com/jitendrarout/ai-vuln-scan>.

References

- [1] PyCQA. *Bandit: A security linter for Python code*. <https://bandit.readthedocs.io>
- [2] ESLint Community. *eslint-plugin-security*. <https://github.com/eslint-community/eslint-plugin-security>
- [3] Semgrep, Inc. *Semgrep: Lightweight static analysis for many languages*. <https://semgrep.dev>